

\paperheight	845.0468pt	29.7000cm
\paperwidth	597.5079pt	21.0000cm
\printpaperheight	845.0468pt	29.7000cm
\printpaperwidth	597.5079pt	21.0000cm
\topspace	27.0086pt	0.9492cm
\backspace	56.9055pt	2.0000cm
\makeupheight	779.8480pt	27.4085cm
\makeupwidth	385.0000pt	13.5312cm
\topheight	0.0000pt	0.0000cm
\topdistance	0.0000pt	0.0000cm
\headerheight	32.6092pt	1.1461cm
\headerdistance	19.1008pt	0.6713cm
\textheight	655.0200pt	23.0213cm
\footerdistance	27.0086pt	0.9492cm
\footerheight	46.1094pt	1.6206cm
\bottomdistance	0.0000pt	0.0000cm
\bottomheight	0.0000pt	0.0000cm
\leftedgewidth	0.0000pt	0.0000cm
\leftedgedistance	0.0000pt	0.0000cm
\leftmarginwidth	75.5820pt	2.6564cm
\leftmargindistance	11.9983pt	0.4217cm
\textwidth	385.0000pt	13.5312cm
\rightmargindistance	14.2264pt	0.5000cm
\rightmarginwidth	110.0000pt	3.8661cm
\rightedgedistance	0.0000pt	0.0000cm
\rightedgewidth	0.0000pt	0.0000cm
\bodyfontsize	10.5400pt	0.3704cm
\lineheight	15.8013pt	0.5554cm
\strutheightfactor	.72	
\strutdepthfactor	.28	
\topskipfactor	1.0	
\maxdepthfactor	0.4	

我们的第一个源文件

本章致力于我们的第一次实验,并将解释 ConT_EXt 文档的基本结构,以及处理潜在错误的最佳策略。

1.1 准备实验:必备工具

要编写并编译第一个源文件,我们需要在系统上安装以下工具。

1. 一个**文本编辑器**,用于编写我们的测试文件。市面上有很多文本编辑器,很难想象哪个操作系统没有预装一个。我们可以使用其中任何一个:有简单的、复杂的、强大的、付费的、免费的(免费啤酒意义上的)、自由的(自由言论意义上的)、专门用于 T_EX 系统的、通用型的等等。如果我们习惯使用某个特定的编辑器,最好继续使用它;如果到目前为止还不习惯使用任何编辑器,我最初的建议是找一个简单的编辑器,以免在学习 ConT_EXt 的难度上再增加学习使用文本编辑器的任务。尽管确实,最难学的程序通常也是最强大的。

本文是用 GNU Emacs 编写的,它是有史以来最强大、最通用的编辑器之一;它确实有自己的特点,也有批评者,但总的来说,“Emacs 爱好者”比“Emacs 反对者”更多。有一个名为 AucTeX 的 GNU Emacs 扩展,用于处理 T_EX 文件或其衍生文件,它为编辑器提供了一些非常有趣的额外实用程序,尽管 AucTeX 通常为处理 L^AT_EX 文件做的准备比处理 ConT_EXt 文件更充分。如果我们不知道选择哪个编辑器,GNU Emacs 结合 AucTeX 可能是一个不错的选择;两者都是自由软件,因此它们有适用于所有操作系统的版本。事实上,说 GNU Emacs 是“自由软件”是一种保守的说法,因为这个程序比其他任何程序都更能体现“自由软件”是什么以及意味着什么的精神。毕竟,它的主要开发者是 RICHARD STALLMAN,他是 GNU 项目和自由软件基金会的创始人和理论家。

除了 GNU Emacs + AucTeX,如果你不知道选择哪个,其他不错的选择有 Scite 和 TexWorks。前者,尽管是一个通用编辑器,并非专门为处理 ConT_EXt 文件而设计,但易于定制,并且由于它是 ConT_EXt 开发者通常使用的编辑器,「ConT_EXt Standalone」包含了这个编辑器的配置文件,由 HANS HAGEN 本人编写。而 TexWorks 是一个快速的文本编辑器,专门处理 T_EX 文件及其衍生语言的文件。将其配置用于 ConT_EXt 非常简单,「ConT_EXt Standalone」也预见了其配置。

无论使用什么编辑器,我们绝对不能用作文本编辑器的是文字处理器,例如 OpenOffice Writer 或 Microsoft Word。这些程序在我看来也过于缓慢和笨重,如

Table of Contents:

- 1.1 准备实验:必备工具
- 1.2 实验本身
- 1.3 我们示例文件的结构
- 1.4 关于如何运行「context」的一些额外细节
- 1.5 错误管理

果明确指定,它们可以将文件保存为“纯文本(txt)”,但它们并非为此而设计,我们最终很可能会将文件保存为与 ConT_EXt 不兼容的某种二进制格式。

2. 一个 ConT_EXt 发行版,用于处理我们的测试文件。如果我们的系统上已经安装了 T_EX (或 L^AT_EX),那么可能已经安装了一个版本的 ConT_EXt。要测试这一点,只需打开一个终端并输入

```
$> context --version
```

注意:对于那些不熟悉终端操作的人,我写的前两个字符(「\$>」)不需要在终端中输入。我只是表示了所谓的终端提示符;那个闪烁的小标志,表示终端正在等待指令。

如果已经安装了某个版本的 ConT_EXt,将会出现类似下面的内容:

```
mtx-context | ConTeXt Process Management 1.06
mtx-context |
mtx-context | main context file: ../context/tex/texmf-context/
               | tex/context/base/mkiv/context.tex
mtx-context | current version: 2026.02.19 11:49
mtx-context | main context file: ../context/tex/texmf-context/
               | tex/context/base/mkx1/context.mkx1
mtx-context | current version: 2026.02.19 11:49
```

最后一行告诉我们安装版本发布的日期。如果这太旧了,我们应该更新它或安装一个新版本。我推荐安装名为「ConT_EXt Standalone」的发行版,其安装说明可以在 [ConT_EXt wiki](#) 上找到。所有这些的摘要可以在附录 ?? 中找到。

3. 一个 PDF 文件阅读器,以便我们在屏幕上看到实验的结果。在 Windows 和 Mac OS 中,通常有 Adobe Acrobat Reader。它不是默认安装的(或者在我 15 多年前停止使用 Microsoft Windows 时不是),但当你第一次尝试打开 PDF 文件时它会安装,所以很可能它已经安装了。Linux/Unix 系统没有当前版本的 Acrobat Reader,但你也不需要它,因为实际上有几十个免费且非常好的 PDF 阅读器可用。此外,这些系统上几乎总是默认安装了其中一个。我最喜欢的是 MuPDF,因为它速度快且易于使用;不过如果你使用带重音字符的英语以外的语言,它有一些缺点,并且不允许你选择文本或将文档发送到打印机;它只是一个阅读器;但它非常快速且使用舒适。当我需要一些 MuPDF 中不可用的功能时,我通常使用 Okular 或 qPdfView。但同样,这是一个品味问题:你可以选择任何你喜欢的。

我们可以选择我们的编辑器、PDF 阅读器、ConT_EXt 发行版.....欢迎来到自由软件的世界!

1.2 实验本身

编写源文件

如果上述工具已经可用,我们需要打开文本编辑器并用它创建一个文件,我们将它命名为「rain.tex」。我们将以下内容写为此文件的内容:

```
% 文档的第一行

\mainlanguage[en] % 语言 = 英语

\setuppapersize[S5] % 纸张大小

\setupbodyfont
[modern,12pt] % 字体 = Latin Modern,12 点

\setuphead % 章节的格式
[chapter]
[style=\bfc]

\starttext % 开始文档内容

\startchapter
[title=西班牙的雨.....]

你真好,让我来。
再说一遍,哪里下雨?
在平原上,在平原上。
那个该死的平原在哪里?
在西班牙,在西班牙。
西班牙的雨主要落在平原上。
西班牙的雨主要落在平原上。

\stopchapter

\stoptext % 文档结束
```

在编写时,更改(几乎)任何内容都无关紧要,尤其是添加或删除空格或换行。重要的是,「\」后面的单词以及花括号内的内容必须完全按照所示编写。其余部分可以有变化。

文件的字符编码

一旦我们写完了上面的内容,我们将文件保存到磁盘上,确保字符编码是 UTF-8(这种字符编码是当今的标准)。无论如何,如果我们不确定,我们可以从文本编辑器本身查看编码,并在需要时更改它。如何做显然取决于我们使用的文本编辑器。例如,在 GNU Emacs 中,按 Ctrl-X,然后按 Return,接着按 『f』,会在窗口底部行(GNU Emacs 称之为小缓冲区)出现一条消息,询问我们新的编码并告诉我们当前的编码;例如,你可能会看到一行如

Coding system for saving file (default utf-8-unix):

这告诉你在保存文件时将使用 UTF-8 字符编码。在其他编辑器中,我们通常可以在「另存为」菜单中找到编码设置。

一旦我们确认编码正确,并将文件保存到磁盘上,我们关闭编辑器,专注于分析我们写的内容。

查看我们的第一个为 ConTeXt 编写的源文件的内容

第一行以「%」字符开头。这是一个保留字符,告诉 ConTeXt 不要处理从该字符到行尾之间的文本。这在我们想在源文件中写入只有作者才能阅读的注释时很有用,因为它不会成为最终文档的一部分。在这个例子中,我使用这个字符来引起对某些行的注意,解释它们的作用。

接下来的行以「\」字符开头,这是 ConTeXt 的另一个保留字符,表示其后的内容是命令的名称。这个示例展示了 ConTeXt 源文件中出现的许多命令:文档所用的语言、纸张大小、文档中将使用的字体以及章节的格式化方式。在后面的其他章节中,我们将看到这些命令的细节,但此刻我只想让读者看到它们的样子:它们总是以「\」开头,然后是命令名称,然后根据情况在花括号(也称为大括号,但在本文档中我们使用“花括号”以明确区别)或方括号内,放置命令产生其效果所需的数据。在命令名称与其后的方括号或花括号之间,可以有空格或一个换行。

在我们示例的第九行(我只计算有文本的行)是重要的 `\starttext` 命令:它告诉 ConTeXt 文档内容从此处开始;而在我们示例的最后一行,我们看到命令 `\stoptext` 表示文档在此结束。这是两个非常重要的命令,我很快会进一步说明。在它们之间是文档的实际内容,在我们的示例中,包括来自「My Fair Lady」「西班牙的雨...」的著名对话。我将其写成散文形式,以便我们可以看到 ConTeXt 如何格式化段落。

处理源文件

下一步,在确认 ConTeXt 已正确安装在我们的系统上之后,我们需要在与源文件「rain.tex」保存的相同目录中打开一个终端。

许多文本编辑器允许我们在不打开终端的情况下编译正在处理的文档。然而,用 ConTeXt 处理文档的规范过程意味着通过直接执行程序从终端进行。在本文档中,我将以这种方式进行(或假定以这种方式进行),原因有几个:首先,我无法知道读者正在使用什么文本编辑器。但最重要的是,通过使用终端,我们将能够访问「context」的屏幕输出,并看到来自程序的消息。

要从终端运行 context,我们必须告诉命令解释器(「shell」)context 在我们系统中的安装位置。这取决于我们选择将 context 安装在哪个地方,但假设我们将其安装在我们的主目录中,那么我们的主目录现在包含一个名为「context」的目录。

在类 Unix 系统(如 Linux)中,命令解释器有一个名为 PATH 的变量,它给出一个目录列表,当尝试执行程序时会搜索这些目录。我们可以将包含 context 程序的目录放在此列表的前面¹,通过输入

```
$> PATH=$HOME/context/tex/texmf-linux-64/bin:$PATH
```

如果我们把「ConTeXt Standalone」安装在其他地方,我们需要相应地调整上面的行。

¹ 我们可能也想把它放在列表末尾,或者中间的某个位置。我们不会在这里争论这个!

为了避免每次打开终端时都输入这一行,可以将上面的行放在 **shell** 的配置文件中。根据我们选择的特定 **shell**,这个配置文件可能是 `$HOME/.bashrc`(如果我们的 **shell** 是 **bash**)、`$HOME/.zshrc`(如果我们的 **shell** 是 **zsh**),或 `$HOME/.profile`(如果我们的 **shell** 是 **sh**)。

如果我们使用的是某种非类 **Unix** 操作系统,我们需要做类似上述的操作。如果我们还不知道怎么做,可以请教隔壁那个无所不知的 10 岁孩子。

一旦我们设置好 **shell** 程序知道如何找到 **context**,我们就可以运行它。我们通过在终端中输入以下命令来实现:

```
$> context rain
```

注意,尽管源文件名为「**rain.tex**」,但在调用「**context**」时我们省略了文件扩展名,因为 **context** 会假定它应该寻找一个扩展名为 **.tex** 的文件。如果我们把源文件命名为例如「**rain.mkiv**」(我经常这样做,以便知道这个文件是为 **Mark IV** 编写的),我们就必须通过输入「**context rain.mkiv**」来明确指定文件扩展名。

在终端中运行「**context**」后,屏幕上会出现几十行文字,告诉我们 **ConTeXt** 正在做什么。这些信息出现的速度人类无法跟上,但我们不必担心,因为除了显示在屏幕上,相同的信息也存储在一个辅助文件中,其扩展名是(在这种情况下,因为我们的 **ConTeXt** 文件名为 **rain.tex**)「**rain.log**」。它是在处理时生成的,如果需要,我们可以稍后从容地查阅它。

几秒钟后,如果我们在源文件中没有犯任何严重错误,终端消息将会结束。最后一条消息会告诉我们编译文件花费了多长时间。第一次编译文档时需要稍多时间,因为 **ConTeXt** 必须将告知它正在使用哪些字体的文件加载到内存中,而后续处理时这些已经加载。当显示耗时信息的最终消息出现时,处理完成。如果一切顺利,我们运行「**context**」的目录现在将包含三个额外的文件:

- **rain.pdf**
- **rain.log**
- **rain.tuc**

第一个是我们处理的结果,或者换句话说,是格式化后的 **PDF**。第二个是「**.log**」文件,存储了处理文件时屏幕上显示的所有信息;第三个是 **ConTeXt** 在编译时生成的辅助文件,用于构建索引和交叉引用。目前,如果一切如预期,我们可以删除这两个文件(**rain.log** 和 **rain.tuc**)。如果存在任何问题,这些文件中的信息将帮助我们找出问题所在,并帮助我们找到解决方案。

如果我们没有得到这些结果,可能是由于以下可能性之一:


```
name: rain.pdf
file: rain.pdf
state: unknown
```

图 1.1 西班牙的雨.....

- ConTeXt 发行版未正确安装。在这种情况下,在终端输入「context」命令后,我们会看到一条包含「command not found: context」等文本的消息。
- 我们的文件不是 UTF-8 编码,导致处理错误。
- 我们在源文件中编写命令名称或其相关数据时出错,或者输入了其他无效的 ConTeXt 输入。

如果在运行「context」后,终端开始发出消息,然后在没有重新出现提示符的情况下停止,我们需要在继续之前按 Ctrl-C 来中止因错误而中断的 ConTeXt 运行。(这在 ConTeXt LMTX 中不太可能发生。)

然后我们需要检查发生了什么,并解决它,直到我们得到正确的编译。

在图 1.1 中,我们看到了「rain.pdf」的内容。我们还看到 ConTeXt 为页面和章节编号,并用我们指示的字体系列书写文本。这里碰巧没有任何单词断字,但默认情况下,ConTeXt 会根据所选语言的断字规则在行尾断字,在我们的例子中,源文件的第一行通过命令 (`\mainlanguage[en]`) 指定了英语。

总结:ConTeXt 转换了源文件并生成了一个文件,其中我们有一个根据源文件中的指令格式化的文档。所有的注释都消失了,就命令而言,我们现在看到的不是它们的名称,而是它们执行的结果。

1.3 我们示例文件的结构

在仅由一个源文件开发的项目中,结构非常简单,由命令 `\starttext ... \stoptext` 标记。从文件的第一行到命令 `\starttext` 之间的所有内容称为导言区。实际文档的内容插入在命令 `\starttext` 和 `\stoptext` 之间。在我们的示例中,导言区包括三个全局配置命令:一个指示文档语言的命令 (`\mainlanguage`),另一个指示页面尺寸的命令 (`\setuppapersize`)(在我们的例子中是「S5」,代表计算机屏幕的尺寸),以及第三个命令 (`\setuphead`),它允许我们配置章节标题的外观。

文档主体由命令 `\starttext` 和 `\stoptext` 界定。这些命令分别表示可处理文本的开始和结束点:在它们之间,我们需要包含所有我们希望 ConTeXt 处理的文本,以及不应影响整个文档而只影响部分文档的命令。目前,我们假设命令 `\starttext` 和 `\stoptext` 在每个 ConTeXt 文档中都是必需的,尽管在后续讨论多文件项目(节 ??)时,我们会看到一些例外情况。

1.4 关于如何运行「context」的一些额外细节

我们可以用各种选项调用「context」。选项紧跟在命令名称之后引入,前面加两个连字符(即减号,而不是任何其他看起来相似的字符)。如果要引入多个选项,我们

用空格分隔它们。在你获得一些专业知识或有某些不寻常的需求之前,你可能不需要使用任何这些选项。但如果你好奇,「**help**」选项会列出所有选项,并附带简要说明:

```
$> context --help
```

一些比较有趣的选项如下:

interface: 正如我在介绍章节中已经说过的,ConTeXt 界面已被翻译成多种语言。默认情况下界面是英文的;但这个选项允许我们告诉它使用捷克语 (**cs**)、荷兰语 (**nl**)、法语 (**fr**)、意大利语 (**it**)、德语 (**de**) 或罗马尼亚语 (**ro**)。

purge, purgeall: 删除处理过程中生成的辅助文件。

result=Name: 指定生成的 PDF 文件的名称。默认情况下,它将与正在处理的源文件相同,但扩展名为「.pdf」。

usemodule=list: 在运行 ConTeXt 之前加载指定的模块(模块是 ConTeXt 的扩展,不是其核心的一部分,提供一些额外的实用功能)。

useenvironment=list: 在运行 ConTeXt 之前加载指定的环境文件(环境文件是包含配置指令的文件)。

version: 显示 ConTeXt 版本。

help: 打印程序选项的帮助信息。注意还有一个「**--expert**」选项,它列出更多的选项。

noconsole: 抑制编译期间通常发送到屏幕的一些消息。不过,这些消息仍保存在「.log」文件中。

在学习 ConTeXt 的早期阶段,我认为使用 **noconsole** 选项不是一个好主意,因为当产生错误时,我们将不知道错误在哪里或是什么导致的(除非查看 **.log** 文件)。亲爱的读者,请相信我,你迟早会在处理过程中遇到错误。

1.5 错误管理

在使用 ConTeXt 时,迟早会在处理过程中出现一些错误,这是不可避免的。我们基本上可以将错误分为以下四类:

1. **书写错误。**当我们弄错命令名称时会产生这些错误。在这种情况下,我们向编译器发送了一个它无法理解的指令。例如,当我们没有写命令 **\TeX** 而是写了 **\Tex**(

最后一个『x』是小写)时,因为 ConT_EXt 区分大小写,因此认为「TeX」和「Tex」是不同的单词。或者,如果命令的功能选项被放在方括号内而不是花括号内,或者我们试图将保留字符当作普通字符使用,等等。

2. **遗漏错误。**在 ConT_EXt 中,有些指令开始一个任务,需要我们明确指示其何时结束;这就像启用数学模式的保留字符『\$』,它一直持续到被禁用,如果我们忘记禁用它,当遇到在数学模式下没有意义的文本或指令时就会产生错误。如果我们以保留字符『{』或 `\startSomething` 命令开始一个文本块,而在后面没有找到明确的结束字符『}』或 `\stopSomething` 命令,也会发生同样的情况。
3. **概念错误。**这是我称之为当调用一个需要某些参数但没有提供这些参数的命令时产生的错误,或者当调用命令的语法不正确时产生的错误。
4. **情境错误。**有些命令被设计为仅在特定上下文或环境中工作,在这些环境之外不被识别。这在数学模式下尤其常见:一些 ConT_EXt 命令仅在编写数学公式时有效,如果在其他环境中调用它们会产生错误。

当「context」在处理过程中警告我们产生了错误时,我们该怎么办? 首先,显然是要确定错误是什么。为此,我们可能需要分析处理过程中生成的「.log」文件,尽管通常这并非必要,因为错误消息将在我们运行「context」的同一个终端中可见。

例如,如果在我们的测试文件「rain.tex」中,错误地将`\starttext`写成了`\startext`(只有一个『t』),这是一个非常常见的错误,当运行「context rain」时,处理将停止,在终端中我们可以看到图 1.2 中显示的信息¹(注意该图只显示了终端输出的末尾,前面会有许多其他“样板”输出行)。在那里我们可以看到源文件的行被编号,在其中一行(这里是第 12 行),编译器在行号和文本之间添加了「>>」,以指示这是它发现错误的行。文件「rain.log」将给我们更多线索。在这个例子中,「rain.log」不是一个大文件,因为被编译的源文件很小;在其他情况下,它可能包含几乎难以应付的大量信息。但我们必须深入研究它。如果我们用文本编辑器打开「rain.log」²,我们会看到它存储了 ConTeXt 正在做的所有事情。我们需要在那里找到以错误消息开头的行,为此我们可以使用文本编辑器的搜索功能。我们将寻找「tex error」,这会带我们找到以下行:

```
tex error > tex error on line 12 in file ./rain-error.tex: |
    Undefined control sequence
<line 3.12>
    \startext
    % Begin document contents
```

注意:「rain.log」文件中告诉我们错误的行很长。为了使这一行在此页面上看起来美观,考虑到本页的宽度,我将其分成了两行。字符『|』表示我分割的位置。

如果我们注意错误消息的三行,我们看到第一行告诉我们是哪一行产生了错误(第 12 行)以及错误的类型:「Undefined control sequence」,也就是:未知的控制序列,换句话说,未知的命令。日志文件接下来的两行向我们显示了第 12 行,并在产生错误的位置进行了分割。因此,毫无疑问错误在于`\startext`。我们仔细阅读,凭借运气和经验,我们会意识到我们写了「startext」而不是「starttext」(有两个『t』)。

想想看,计算机非常擅长且非常快速地执行指令,但非常不擅长读取我们的思想,单词「startext」与「starttext」不同。程序知道如何执行后者,而不是前者。它不知道如何处理那个。

其他时候,找到错误不会那么容易。特别是当错误在于某事物已经开始但未指示其必须结束的位置时。有时,我们不应该在「.log」文件中搜索「tex error」,而应该寻找一个星号。文件中行首的这个字符与其说是致命错误,不如说是一个警告。然而,警告有助于找到错误。

如果「.log」文件中的信息不够,我们需要逐位检查我们的主文件,寻找错误。一个很好的策略是更改`\stoptext`命令的位置:记住 ConTeXt 在遇到此命令时会停止处理文本。因此,如果我将一个`\stoptext`大致放在文件中间并编译,只有前半部分会被编译;如果错误再次出现,那么我知道它在源文件的前半部分,如果没有,则意味着它在后半部分... 如此类推,通过逐位更改`\stoptext`命令的位置,我们将能够

```
3 \setuppapersize[S5]
% 纸张大小
4
5 \setupbodyfont
6 [modern,12pt] %
字体 = Latin Modern,12
点
7
8 \setuphead % 章
节的格式
9 [chapter]
10 [style=\bfc]
11
12 >> \startext % 开始
文档内容
13
14 \startchapter
15 [title=西班牙的
雨.....]
16
17 你真好,让我来。
18 再说一遍,哪里下
雨?
19 在平原上,在平原
上。
20 那个该死的平原
在哪里?
21 在西班牙,在西班
牙。
22 西班牙的雨主要
落在平原上。
错误消息顶部行末尾的
控制序列从未被 \def'ed。
你可以继续,我会忘记任
何未定义的东西。
mtx-context | fatal
error: return code: 1
```

图 1.2 编译错误情况下的屏幕输出

¹ 注意:错误报告的确切格式可能会随时间变化,但你应该会看到类似的信息。

² 如果你的文本编辑器能够自动搜索并显示此错误消息,如果你已经“升级”到从编辑器内部运行 ConTeXt,那就可以。例如,如果你使用带有「AUCTeX」包的 Emacs,并使用「Ctrl-C Ctrl-C」让 Emacs 为你编译文档,那么输入「Ctrl-C」会将你的编辑窗口分成两部分:一部分会显示错误消息,而另一部分会显示有问题的 ConTeXt 代码行。

找到错误所在。一旦找到它,我们就可以尝试理解并纠正它,或者,如果我们无法理解为什么会产生错误,至少通过找到它所在的位置,我们可以尝试以另一种方式编写以避免重现它。

在实践中,当用 ConTeXt 制作相对较长的文档时,通常会在起草文档的过程中不时编译,这样如果它“抛出”错误,我们会大致清楚自上次处理文件以来我们添加或更改了什么,以及为什么 ConTeXt 会抛出错误。换句话说,通常建议在创建和/或编辑 ConTeXt 文档时,每隔几分钟重新编译一次。